

Jobsheet 14

Nama : Lindhu Nuril Rahmatdanto

NIM : 254107020216

Kelas : TI 1G

1. Mengapa dalam binary search tree proses pencarian data bisa lebih efektif dilakukan dibanding binary tree biasa?

Karena pada Binary Search Tree terdapat aturan khusus:

Semua node di kiri memiliki nilai lebih kecil dan semua node di kanan memiliki nilai lebih besar

Karena ada aturan urutan tersebut, proses pencarian tidak perlu mengecek semua node satu per satu.

2. Untuk apakah di class Node, kegunaan dari atribut left dan right? atribut tersebut digunakan untuk menyimpan referensi/pointer ke anak node right untuk kanan dan left untuk kiri

3. a. Untuk apakah kegunaan dari atribut root di dalam class BinaryTree? Menjadi titik awal semua proses tree, juga digunakan saat pencarian ,penambahan dll

b. Ketika objek tree pertama kali dibuat, apakah nilai dari root?
root = null

4. Ketika tree masih kosong, dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?

Akan terjadi pengecekan apakah tree kosong atau tidak, apabila kosong maka newnode akan menjadi root.

5. Perhatikan method add(), di dalamnya terdapat baris program seperti di bawah ini. Jelaskan secara detil untuk apa baris program tersebut?

node curen akan disimpan ke parent agar program tahu dari node

berikutnya. Dalam kondisi If terdapat pengecekan apakah IPK node baru lebih kecil dari node saat ini, apabila iya maka node current berpindah ke current left, apabila kosong maka akan ditempati. Pada kondisi If terjadi proses serupa hanya saja

pindahanya ke kanan

6. Jelaskan langkah-langkah pada method delete() saat menghapus sebuah node yang memiliki dua anak. Bagaimana method getSuccessor() membantu dalam proses ini?

Apabila node mempunyai 2 anak BST tidak bisa langsung menghapus node tersebut karena tree bisa rusak, maka solusinya adalah memakai successor. Successor akan menjadi tumbal yang menggantikan node yang dihapus.

1. Apakah kegunaan dari atribut data dan idxLast yang ada di class BinaryTreeArray?

atribut data digunakan untuk menyimpan kumpulan data jadi 1 data

atribut idxLast digunakan untuk menyimpan index terakhir yang berisi data

2. Apakah kegunaan dari method populateData()?

Digunakan untuk memasukkan data array mahasiswa ke binary tree dan menentukan index terakhir data

3. Apakah kegunaan dari method traverseInOrder()?

Untuk menampilkan isi binary tree dengan metode InOrder()

4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?

index kiri = 5 sedangkan kanan = 6

5. Apa kegunaan statement `int idxLast = 6` pada praktikum 2 percobaan nomor 4?

Untuk menandakan index terakhir yang berisi data.

6. Mengapa indeks $2 * idxStart + 1$ dan $2 * idxStart + 2$ digunakan dalam pemanggilan rekursif, dan apa kaitannya dengan struktur pohon biner yang disusun dalam array?

Karena array tidak memiliki pointer seperti linked tree. Maka rumus seperti ini diperlukan

```

if (dataMahasiswa[idxStart] != null) {
    traverseInOrder(2*idxStart+1);
    dataMahasiswa[idxStart].tampilInformasi();
    traverseInOrder(2*idxStart+2);
}

```

Laporan Praktikum

```

package Jobsheet14;

public class BinaryTree17 {
    Node17 root;

    BinaryTree17(){
        root = null;
    }
    public boolean isEmpty(){
        return root == null;
    }
    public void add(Mahasiswa17 mahasiswa){
        Node17 newnode = new Node17(mahasiswa);
        if (isEmpty()) {
            root = newnode;
        }else{
            Node17 current = root;
            Node17 parent = null;
            while (true) {
                parent = current;
                if (mahasiswa.ipk < current.mahasiswa.ipk) {
                    current = current.left;
                    if (current == null) {
                        parent.left = newnode;
                        return;
                    }
                }else{
                    current = current.right;
                    if (current == null) {
                        parent.right = newnode;
                        return;
                    }
                }
            }
        }
    }

    boolean find(double ipk){
        boolean result = false;
    }
}

```

```

Node17 current = root;
while (current != null) {
    if (current.mahasiswa.ipk == ipk) {
        result = true;
        break;
    }else if (ipk > current.mahasiswa.ipk) {
        current = current.right;
    }else{
        current = current.left;
    }
}
return result;
}

void traversePreOrder(Node17 node){
    if (node != null) {
        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}

void traverseInOrder(Node17 node){
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

void traversePostOrder(Node17 node){
    if (node != null) {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}

Node17 getSucessor(Node17 del){
    Node17 sucessor = del.right;
    Node17 sucessorParent = del;
    while (sucessor.left != null) {
        sucessorParent = sucessor;
        sucessor = sucessor.left;
    }
    if (sucessor != del.right) {
        sucessorParent.left = sucessor.right;
        sucessor.right = del.right;
    }
}

```

```

    }
    return sucessor;
}
void delete(double ipk){
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }
    Node17 parent = root;
    Node17 current = root;
    boolean isLeftChild = false;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            break;
        }else if (ipk < current.mahasiswa.ipk) {
            parent =current;
            current = current.left;
            isLeftChild = false;
        }else if (ipk > current.mahasiswa.ipk) {
            parent = current;
            current = current.right;
            isLeftChild = false;
        }
    }
    if (current == null) {
        System.out.println("Data tidak ditemukan");
        return;
    }else{
        if (current.left == null && current.right == null) {
            if (current == root) {
                root = null;
            }else {
                if (isLeftChild) {
                    parent.left = null;
                }else{
                    parent.right = null;
                }
            }
        }
        }else if (current.left == null) {
            if (current == root) {
                root = current.right;
            }else{
                if (isLeftChild) {
                    parent.left = current.right;
                }
            }
        }
    }
}

```

```

        }else{
            parent.right = current.right;
        }
    }
}
}else if (current.right == null) {
    if (current == root) {
        root = current.left;
    }else{
        if (isLeftChild) {
            parent.left = current.left;
        }else{
            parent.right = current.left;
        }
    }
}
}else if (current.right == null) {
    if (current == root) {
        root = current.left;
    }else{
        if (isLeftChild) {
            parent.left =current.left;
        }else{
            parent.right =current.left;
        }
    }
}
}else{
    Node17 sucessor = getSucessor(current);
    System.out.println("Jika 2 anak, current = ");
    sucessor.mahasiswa.tampilInformasi();
    if (current == root) {
        root = sucessor;
    }else{
        if (isLeftChild) {
            parent.left = sucessor;
        }else {
            parent.right = sucessor;
        }
    }
    sucessor.left =current.left;
}
}
}

void addRekursif(Mahasiswa17 mahasiswa){
    root = addRekursif(root, mahasiswa);
}
}

```

```

Node17 addRekursif(Node17 current, Mahasiswa17 mahasiswa){
    if (current == null) {
        return new Node17(mahasiswa);
    }

    if (mahasiswa.ipk < current.mahasiswa.ipk) {
        current.left = addRekursif(current.left, mahasiswa);
    }else{
        current.right = addRekursif(current.right, mahasiswa);
    }

    return current;
}

void cariMinIPK(){
    if (isEmpty()) {
        System.out.println("Tree kosong");
        return;
    }

    Node17 current = root;

    while (current.left != null) {
        current = current.left;
    }

    System.out.println("Mahasiswa dengan IPK terkecil : ");
    current.mahasiswa.tampilInformasi();
}

void cariMaxIPK(){
    if (isEmpty()) {
        System.out.println("Tree kosong");
        return;
    }

    Node17 current = root;

    while (current.right != null) {
        current = current.right;
    }

    System.out.println("Mahasiswa dengan IPK terbesar : ");
    current.mahasiswa.tampilInformasi();
}

```

```

}
void tampilMahasiswaIPKDiAtas(double ipkBatas){
    tampilMahasiswaIPKDiAtas(root, ipkBatas);
}

void tampilMahasiswaIPKDiAtas(Node17 node, double ipkBatas){
    if (node != null) {

        tampilMahasiswaIPKDiAtas(node.left, ipkBatas);

        if (node.mahasiswa.ipk > ipkBatas) {
            node.mahasiswa.tampilInformasi();
        }

        tampilMahasiswaIPKDiAtas(node.right, ipkBatas);
    }
}
}

```

```

package Jobsheet14;

public class BinaryTreeMain17 {
    public static void main(String[] args) {
        BinaryTree17 bst = new BinaryTree17();

        bst.add(new Mahasiswa17("244160121", "Ali", "A", 3.57));
        bst.add(new Mahasiswa17("244160221", "Badar", "B", 3.85));
        bst.add(new Mahasiswa17("244160185", "Candra", "C", 3.21));
        bst.add(new Mahasiswa17("244160220", "Dewi", "B", 3.54));

        System.out.println("Daftar Semua Mahasiswa (in order traversal):");
        bst.traverseInOrder(bst.root);

        System.out.println("Pencarian data mahasiswa : ");
        System.out.println("Cari Mahasiswa dengan ipk : 3.54");
        String hasilcari = bst.find(3.54)? "Ditemukan" : "Tidak ditemukan";
        System.out.println(hasilcari);

        System.out.println("Cari mahasiswa dengan ipk: 3.22 ;");
        hasilcari = bst.find(3.22)? "Ditemukan" : "Tidak ditemukan";
    }
}

```



```

        System.out.println(hasilcari);

        bst.add(new Mahasiswa17("244160131", "Devi", "A", 3.72));
        bst.add(new Mahasiswa17("244160205", "Ehsan", "D", 3.37));
        bst.add(new Mahasiswa17("244160170", "Firi", "B", 3.46));
        System.out.println("Daftar semua mahasiswa setelah penambahan 3
mahasiswa");
        System.out.println("InOrder Traversal");
        bst.traverseInOrder(bst.root);
        System.out.println("PreOrder Traversal");
        bst.traversePreOrder(bst.root);
        System.out.println("PostOrder Traversal");
        bst.traversePostOrder(bst.root);

        System.out.println("Penghapusan data mahasiswa");
        bst.delete(3.57);
        System.out.println("Daftar semua mahasiswa setelah penghapusan 1
mahasiswa (InOrder Traversal)");
        bst.traverseInOrder(bst.root);

        System.out.println("Tambah data rekursif");
        bst.addRekursif(new Mahasiswa17("244160300", "Rama", "A", 3.90));

        System.out.println("InOrder Traversal");
        bst.traverseInOrder(bst.root);

        System.out.println("IPK minimum");
        bst.cariMinIPK();

        System.out.println("IPK maksimum");
        bst.cariMaxIPK();

        System.out.println("Mahasiswa dengan IPK di atas 3.50");
        bst.tampilMahasiswaIPKDiAtas(3.50);
    }
}

```

```

package Jobsheet14;

```

```

public class BinaryTreeArray17 {
    Mahasiswa17 [] dataMahasiswa;
    int idxLast;

    public BinaryTreeArray17(){
        this.dataMahasiswa = new Mahasiswa17[10];
    }

    void populateData (Mahasiswa17 dataMhs [], int idxLast){
        this.dataMahasiswa = dataMhs;
        this.idxLast = idxLast;
    }

    void traverseInOrder(int idxStart){
        if (idxStart <= idxLast) {
            if (dataMahasiswa[idxStart] != null) {
                traverseInOrder(2*idxStart+1);
                dataMahasiswa[idxStart].tampilInformasi();
                traverseInOrder(2*idxStart+2);
            }
        }
    }

    void add(Mahasiswa17 data){
        if (idxLast == dataMahasiswa.length - 1) {
            System.out.println("Array penuh");
            return;
        }

        idxLast++;

        dataMahasiswa[idxLast] = data;
    }

    void traversePreOrder(int idxStart){
        if (idxStart <= idxLast) {
            if (dataMahasiswa[idxStart] != null) {

                dataMahasiswa[idxStart].tampilInformasi();

                traversePreOrder(2 * idxStart + 1);

                traversePreOrder(2 * idxStart + 2);
            }
        }
    }
}

```

```
}  
}
```

```
package Jobsheet14;  
  
public class BinaryTreeArrayMain17 {  
    public static void main(String[] args) {  
        BinaryTreeArray17 bta = new BinaryTreeArray17();  
        Mahasiswa17 mhs1 = new Mahasiswa17("244160121", "Ali", "A", 3.57);  
        Mahasiswa17 mhs2 = new Mahasiswa17("244160185", "Candra", "C", 3.41);  
        Mahasiswa17 mhs3 = new Mahasiswa17("244160221", "Badar", "B", 3.75);  
        Mahasiswa17 mhs4 = new Mahasiswa17("244160220", "Dewi", "B", 3.35);  
  
        Mahasiswa17 mhs5 = new Mahasiswa17("244160131", "Devi", "A", 3.48);  
        Mahasiswa17 mhs6 = new Mahasiswa17("244160205", "Ehsan", "D", 3.61);  
        Mahasiswa17 mhs7 = new Mahasiswa17("244160170", "Firi", "B", 3.86);  
  
        Mahasiswa17[] dataMahasiswa =  
{mhs1,mhs2,mhs3,mhs4,mhs5,mhs6,mhs7,null,null,null};  
        int idxLast = 6;  
        bta.populateData(dataMahasiswa, idxLast);  
        System.out.println("InOrder Traversal Mahasiswa: ");  
        bta.traverseInOrder(0);  
  
    }  
}
```